

MyDelivery Platform: Fresher Onboarding Guide

Welcome! This document gives you a beginner-friendly overview of three related applications plus shared components and support assets:

1. MyDelivery (Customer-facing redelivery & self-service portal)
2. MyDelivery Admin (Operational / back-office tooling for managing delivery/redelivery rules & data)
3. B2C (Business-to-Consumer notification & alerting services)
4. Shared Components (Reusable domain logic & integration helpers)
5. Support Folders (Project documents: requirements, architecture, deployment, test artifacts)

The goal: help you understand what each part does, where to look for code, and how requests flow through the system.

1. MyDelivery Application

What Problem It Solves

Allows end customers to manage parcel redelivery: choose an option (deliver to original address, alternative address, neighbour, leave in absence, collect from depot), select dates, confirm details, and optionally print forms/confirmations.

Main Technologies (Legacy Stack)

- Spring WebFlow (wizard-style UI state machine) + Tiles / Freemarker templates
- Spring MVC (WebService module – machine/API endpoints)
- Java (pre-Java 17), Oval for validation, JasperReports / StreamServe for PDFs
- External service adapters ("ACL" classes) for systems like APC, OSC, CommonCodes, StreamServe

High-Level Flow (Customer Journey)

1. User lands on index.jsp -> forwarded to WebFlow `redelivery-flow.xml`.
2. Flow initializes (loading country + consignment id display info).
3. User enters consignment & basic details (or skips if coming from emailed link).
4. System calls service layer to calculate available redelivery options.
5. User selects an option; system fetches allowed dates for that specific option.
6. User enters contact & address details (varies by option).
7. Confirmation step: service builds a validated RedeliveryRequest, returns a confirmation model.
8. Optional: user prints PTL form or confirmation details (PDF).
9. Flow ends (redirect to external site).

Key Code Areas

- Presentation module: Web UI, decorators, converters, validation for forms.
- ServiceImpl module: Business rules, option eligibility, date calculations, report building.
- ServiceApi module: Interfaces & model contracts shared across layers.
- WebService module: API endpoints for external/internal clients (e.g., mobile).

Common Terms

- Consignment: Shipment entity (parcel or set of parcels) the user wants to redeliver.
- Redelivery Option: Mode of redelivery (alternative address, self collection, etc.).
- Decorator: Object that prepares data for a particular step in the UI flow.
- ACL (Anti-Corruption Layer): Wrapper around external systems so internal domain stays clean.
- Model: Data object passed between layers; often has an API interface and a "Default" implementation.

Where to Start Debugging

1. Reproduce a user flow: run Presentation module, set a breakpoint in `redelivery-flow.xml` (decorator methods referenced) or in `DefaultMyDeliveryService`.
2. For API calls: start WebService module and hit an endpoint returning consignment details; trace into ACL calls.
3. External systems: find respective `Default*AcL` classes.

2. MyDelivery Admin Application

Purpose

Provides administrative UI/tools for internal staff (support / operations) to manage configurations: country rules, option enablement, depot data, system parameters, maybe dispute/override actions.

Typical Features (Based on Naming & Standard Patterns)

- View & edit redelivery rules (e.g., enable / disable `LeaveWithNeighbour` for a country)
- Manage depot opening times / addresses
- Monitor redelivery request statuses / audit logs
- Trigger manual report generation or resend emails

Technology Similarities

- Also multi-module (AdminPresentation, AdminService, etc.)
- Likely uses similar Spring + JSP/Freemarker + possibly WebFlow or standard controllers
- Shares some domain models / utilities via cross-dependencies modules

Key Modules (Folder Names)

- `MyDeliveryAdminPresentation` : UI layer.

- `MyDeliveryAdminApplicationLayer` / `MyDeliveryAdminCrossDependencies` : Shared logic & integration adaptors.
- `MyDeliveryAdminEAR` : Packaging for deployment.
- `MyDeliveryAdminConfig` : Environment-specific configuration (XML, property files).

How It Integrates

- Reuses components from `eai-3532120-mydelivery-components` (domain and service APIs) for consistency.
- Calls external systems via similar ACL patterns to fetch/update admin data (e.g., depot lists).

Fresher Tips

- Look for controllers/servlets in `AdminPresentation` to understand entry points.
- Compare Admin service classes with MyDelivery equivalent to see extended functionality.
- Identify differences: Admin flows emphasize CRUD & configuration, not the customer wizard.

3. B2C (Business-To-Consumer) Application

Purpose

Handles outbound notification workflows: sending alerts/emails/SMS to consumers regarding shipment status changes, redelivery reminders, or marketing communications.

Main Modules (Example: B2C-NotificationService)

- Job configurations (`process-alerts-job.xml` , `process-consignment-statuses-job.xml`) – scheduled tasks scanning for events.
- Properties folder with alerting configurations (frequency thresholds, message templates).
- Notification sending logic (likely uses email/SMS connectors).
- Integration tests verifying end-to-end message dispatch.

High-Level Operational Flow

1. Scheduled job runs (cron-like configuration) reading consignment status changes or pending alerts.
2. Batch components load eligible events (e.g., out-for-delivery, delivery failed, ready for collection).
3. Rules filter which customers receive which notifications (frequency caps, opt-outs).
4. Template engine fills placeholders (names, dates, depot info).
5. Message dispatch via communication channels (email/SMS).
6. Status recorded (success/failure) -> potential retry logic.

Key Concepts

- Alert Job: A scheduled process scanning for new events.

- Template: A message pattern with variables replaced at runtime.
- Channel: Medium of communication (EMAIL, SMS, maybe PUSH).
- Retry / Dead Letter: Handling failed sends (log & reattempt).

Fresher Tips

- Start at `process-alerts-job.xml` to see job bean definitions.
- Locate service classes in `src/main/java` of B2C-NotificationService for event retrieval & message generation.
- Review property files for feature toggles (turning templates or channels on/off).

4. Shared Components (eai-3532120-mydelivery-components)

Purpose

Central library of reusable code so MyDelivery, Admin, and B2C do not duplicate logic.

Typical Contents (Based on Folder Names)

- `delivery-domain-properties` : Common domain property models.
- `delivery-async-process` : Shared asynchronous processing utilities.
- `entity-api` / `entity-impl` : Domain entities & their implementations.
- `service-api` / `service-impl` : Shared business service contracts & default logic.
- `delivery-domain-enquiry` : Specialized query/reporting logic.
- `MDLocationService` : Location lookups (geocoding, depot resolution).

How To Use

- Applications add these modules as Maven dependencies.
- Import interfaces from `entity-api` and instantiate implementations from `entity-impl` or via Spring.

Benefits

- Consistent domain definitions (one Consignment model, one Depot entity).
- Reduced maintenance (fix bug once, all apps benefit).
- Easier modernization (upgrade Java version in one place).

5. Support Folders (eai-3532120-mydelivery-support & B2C support)

Contain project management & lifecycle documentation:

- Requirements (CSR docs, Business Justification)
- Architecture & design (A&D) diagrams
- Test plans & cases

- Deployment guides & environment configuration notes
- Estimation spreadsheets & release planning documents

Fresher Usage

- Read requirements to understand business rules before editing code.
- Reference design docs for data flow diagrams.
- Use test folder artifacts to see expected behaviors and edge cases.

6. Common Patterns Across All Apps

Pattern	Description	Fresher Hint
Layered Architecture	Presentation → Service → Integration/ACL → External Systems	Trace a request top-down starting from controller or flow definition.
Model + Implementation	Interface (in API module) & Default implementation (in Impl module)	Open interface first to learn contract.
Validation	Custom annotations / helpers or legacy OVal; migrating to Jakarta Validation recommended	Keep fields clean, add appropriate annotations.
External Adapters (ACL)	Encapsulate remote API calls & data mapping	If adding new external system, copy pattern.
Reporting	Builder + Labels + Printer classes for PDF creation	Start with builder to see required fields.
Configuration	Properties / XML context; moving to application.yml under Spring Boot	Avoid hardcoding; use config values.

7. Getting a Local Dev Environment Running (Simplified)

1. Import Maven multi-module project into IDE.
2. Run a clean build: `mvn clean install` at root (may need corporate repos configured).
3. Start MyDelivery Presentation (embedded server or app server) – if legacy, deploy EAR to WebSphere/Tomcat.
4. Access `http://localhost:<port>/` → redelivery flow.
5. For Admin: deploy Admin EAR; access admin UI base URL.
6. For B2C Notification jobs: run Spring context that defines scheduled jobs or trigger main class if available.

8. Debugging Tips

- Use breakpoints in service impl classes (`DefaultMyDeliveryService`) for option logic.
 - Add logging around ACL calls if external data seems wrong.
 - For date issues: check conversions & time zone (migrate to `java.time` APIs early).
 - When validation fails: inspect `ViolationsModel` or equivalent; look at field length constants.
 - For B2C jobs not firing: ensure scheduling config loaded & necessary properties set.
-

9. Modernization Roadmap (Shared Goals)

Short-Term:

- Replace `SimpleDateFormat` with `DateTimeFormatter` .
- Introduce unit tests for core helpers (option eligibility, template rendering, depot selection).
- Add README per module summarizing purpose.

Mid-Term:

- Migrate to Java 17 + Spring Boot for unified configuration.
- Consolidate validation to Jakarta Validation (Hibernate Validator).
- Introduce REST endpoints for flow steps (phase out WebFlow if desired).
- Implement caching via Spring Cache + Caffeine/Redis.

Long-Term:

- Containerize all apps, use CI/CD pipeline.
 - Introduce observability (Micrometer metrics, OpenTelemetry tracing).
 - Refactor ACLs to resilient clients (Resilience4j for retries & circuit breakers).
 - Replace JasperReports + StreamServe integration with simplified PDF service or front-end rendering if feasible.
-

10. How To Ask Good Questions (Team Culture)

- Identify module & class where you are stuck.
 - Describe input, expected output, and actual output.
 - Share relevant log snippets (avoid sensitive data).
 - Propose a hypothesis ("I think X fails because Y").
-

11. Quick Glossary

Term	Meaning
Consignment	Parcel shipment record.
Depot	Physical location for parcel collection.
Redelivery	Scheduling a new delivery attempt / alternative handling.
Option	A user-selectable redelivery method.
ACL	Anti-Corruption Layer (external service adapter).
Model	Data container for request or response.
Decorator	Prepares view/model for a UI step.
Validation	Checks input meets business rules.
Record (future)	Java 17 concise data carrier class.

12. First Week Suggested Tasks

Day 1: Read this guide + skim support requirements docs. Day 2: Run MyDelivery locally, walk through option selection. Day 3: Trace code path for obtaining available options. Day 4: Explore B2C notification job configuration. Day 5: Pair with senior on adding a small validation rule.

13. Safety & Quality Reminders

- Never commit secrets (passwords, keys). Use environment variables.
 - Add tests for any new logic (unit + integration where possible).
 - Keep methods small; prefer clear naming over comments.
 - Log at INFO for significant lifecycle events; DEBUG for detailed tracing.
-

14. Next Steps

After you understand basics, review the Java 17 migration document (`MyDeliveryFlowAndJava17Migration.md`) for future direction. Contribute improvements by creating small pull requests and tagging mentors for review.

Welcome aboard! 🚀